

Dzień 1

Docker + Kubernetes podstawy

Maciej Krajewski · 2026

Plan dnia

1. **Docker**: image, optymalizacja Dockerfile, hardening, skanowanie
2. **K8s podstawy**: API, Pod, Service, Probes, Deployment, Namespace
3. **kubectl**: pierwsze komendy, context, debug
4. **Multi-container Pod**: sidecar pattern

→ Repo: `day1/`

Sekcja 1.1 — Dlaczego Docker?

- Konteneryzacja = uniformność środowiska
- "Działa u mnie" → "działa wszędzie"
- Odseparowanie aplikacji od OS
- Lekkie (vs VM): startup ms, overhead minimalny

Optymalizacja Dockerfile (D1/03)

Naiwny	Zoptymalizowany
FROM node:20 (1.2 GB)	FROM node:20-alpine AS build + distroless final
COPY . przed npm install	COPY package*.json first → cache layer
apt-get install vim git ...	tylko runtime deps
1 stage	multi-stage

Efekt: 1200 MB → 150 MB, build time -60%

→ `day1/03_dockerfile_optimization/`

Skanowanie obrazów: Trivy (D1/04)

```
trivy image --severity HIGH,CRITICAL --exit-code 1 myapp:v1
```

- OS packages CVE (apt/apk/yum)
- Language deps (npm, pip, go.mod)
- Misconfig (Dockerfile, K8s YAML)
- Secrets w obrazie

W CI: aquasecurity/trivy-action z exit-code → blokuje merge

→ day1/04_image_scanning_trivy/

Podpisywanie obrazów: Cosign + SBOM (D1/05)

```
cosign sign --key cosign.key myapp:v1  
cosign attest --predicate sbom.json --type cyclonedx myapp:v1  
cosign verify --key cosign.pub myapp:v1
```

- **Keyless** (OIDC) lepsze dla CI — krótko żyjące tożsamości
- **SBOM** = Software Bill of Materials (lista wszystkich komponentów)
- **Rekor** = transparency log (publiczny audit trail)

→ `day1/05_image_signing_cosign/`

Hardening obrazu (D1/06)

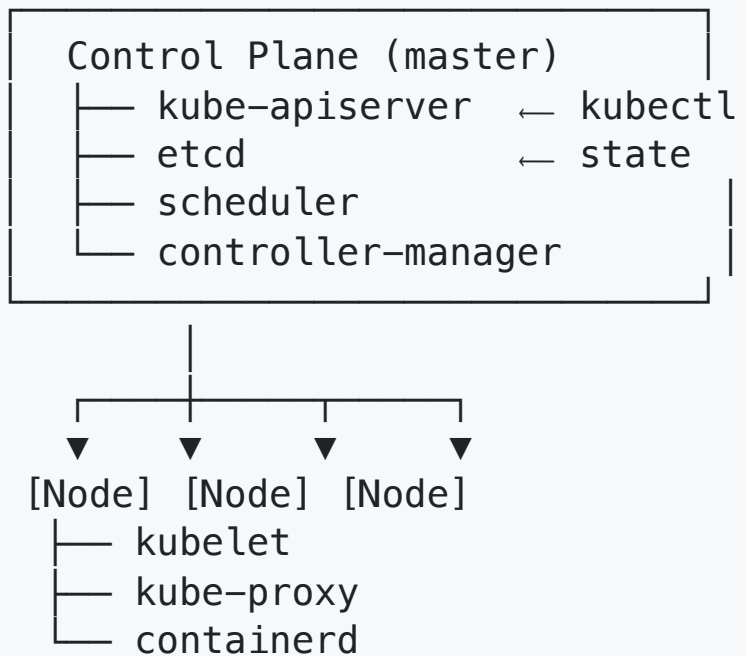
Checklist:

-  Distroless lub Alpine base
-  `USER nonroot`
-  Pinowane wersje (`nginx:1.27.0-alpine`)
-  Multi-stage build
-  `.dockerignore`
-  `HEALTHCHECK`
-  Hadolint w CI
-  Trivy + Cosign

Obraz musi być bezpieczny *nawet jeśli* uruchomi się bez SecurityContext klastra

→ `day1/06_hardening/`

Sekcja 1.2 — Architektura K8s



→ `day1/04_k8s/`

Pod = najmniejsza jednostka

- 1 lub więcej kontenerów
- Współdzielona sieć (localhost)
- Współdzielone wolumeny
- **Efemeryczny** — restart = nowy IP

```
kubectl apply -f pod.yaml  
kubectl get pods  
kubectl exec -ti myapp -- sh  
kubectl logs -f myapp  
kubectl port-forward pod/myapp 8080:80
```

→ day1/05_pod/

Multi-container Pod: sidecar (D1/13)

Wzorce:

- **Sidecar** — log shipper, metrics scraper
- **Ambassador** — local proxy do external service
- **Adapter** — format normalizer
- **Init container** — pre-start setup (D3/01)

Native sidecar (K8s 1.29+): `initContainers + restartPolicy: Always`

→ `day1/13_multi_container_pod/`

Probes: Liveness, Readiness, Startup (D1/09)

Probe	Fail action	Kiedy
Liveness	restart kontenera	deadlock
Readiness	usuń ze Service Endpoints	warming up
Startup	tłumi inne probe	wolne JVM apps

Antywzorzec: Liveness == Readiness — może spowodować boot loop

→ `day1/09_probes/`

Deployment + rolling update (D1/11)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
```

- ReplicaSet pod spodem (history dla rollback)
- `kubectl rollout status / history / undo`
- Zmiana template → nowy ReplicaSet, stary → 0

Strategie wdrożeń (porównanie 6): D3/05 strategies.md

→ `day1/11_deployment/`

kubectl context (D1/14)

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context k3d-training
kubectl config set-context --current --namespace=myapp

# Bezpiecznie: explicit context per komenda destruktywna
kubectl --context=production delete pod ...
```

Slajd narzędzia (na końcu D5): kubectlx, kubens, k9s, kube-ps1

→ `day1/14_kubectl_context/`

Podsumowanie D1

- ✓ Docker: optymalizacja + hardening + Trivy + Cosign
- ✓ K8s: API, Pod, Service, Deployment, Namespace
- ✓ kubectl basics + context management
- ✓ Multi-container patterns

Domowa praca:

- Zbuduj swój obraz (D1/03 + 06)
- Skanuj Trivy (D1/04)
- Wdroż na lokalnym K3d (`./setup-cluster.sh`)

Jutro: Workloady, ConfigMaps, Secrets, AuthN/AuthZ, Gateway API